

Compiler Construction

Lab 02: A lexical analyzer in C

Section 59 B & G

Your working directory is `sec_59_b_and_g/labs/lab_02/`. Use the public repository named `compiler-demo`. The instructor will announce its URL, task submission window, task freeze time, assignment extension time and viva arrangements. All announced local times are Bangladesh time unless the instructor says otherwise.

Part 1 - Objective

You will read a C source file as a character stream, find its lexemes and print their token categories. You will also list the complete token sequence of your section's `public.c` by hand in `tokens.md`.

Your two submitted files are `tokens.md` and `lexical_analyzer.c`. The analyzer must work on other input files that follow the rules below. AI tools are allowed. You must understand every submitted part and explain it in the viva.

Part 2 - Forking the Public Repository

1. Sign in to your own student GitHub account. Check the profile menu before continuing.
2. Open the instructor's `compiler-demo` URL. Click **Fork**. Select your account as the owner. Keep the repository name. Use the default `main` branch and create the fork.
3. Fork only once for this course. If you already have your course fork, open that fork instead of creating another one.
4. In your fork, click **Code** and copy your fork's clone URL. Clone it using your own student account's authentication. In a terminal, run the following after replacing `STUDENT` with your username:

```
git clone https://github.com/STUDENT/compiler-demo.git
cd compiler-demo
git switch main
git remote -v
```

5. Verify that `origin` points to your student fork. Add the instructor repository as `upstream` if it is not already present; replace `INSTRUCTOR`:

```
git remote add upstream https://github.com/INSTRUCTOR/compiler-demo.git
git fetch upstream
```

6. Configure your commit name and email inside this clone. A commit identity is separate from the GitHub account used to authenticate a push:

```
git config user.name "Your Full Name"
git config user.email "YOUR_GITHUB_EMAIL"
git config user.name
git config user.email
```

7. If this is an existing semester fork, save any unfinished local work first. Run `git fetch upstream`, then `git merge upstream/main` while on `main`. Resolve any conflict without deleting your earlier work. This fetch/merge is local; do not push a Lab 02 solution early. Ask the instructor if your fork contains unrelated changes that would enter the new PR.

Part 3 - Finding Your Section

1. Open the clone in your editor. Find `sec_59_b_and_g` and then `labs/lab_02`.
2. Open `Lab_02.pdf` and `public.c` in that directory. This is your question. Do not use the question from `sec_59_e_and_f`.
3. Read the source below and the lexical contract in Part 6. Do not modify `public.c`, either section's PDF, the repository README, or the other section's files.

```
int main() {
    int count = 3;
    int total = 0;
    // Add the remaining count.
    while (count > 0) {
        total = total + count;
        count = count - 1;
    }
    if (total >= 6 && count == 0) {
        return total;
    } else {
        return 0;
    }
}
```

Part 4 - Preparing Your Submission Directory

1. From the clone root, go into your Lab 02 directory:

```
cd sec_59_b_and_g/labs/lab_02
```

2. In the editor/file manager, rename `submissions/your-id` to your real university student ID. Use the format `NNN-NNN-NNN`, for example `232-115-001`. Keep the two filenames unchanged. Use your real ID for assessed work, not the example ID.
3. Open the renamed directory. Confirm that it contains `tokens.md` and `lexical_analyzer.c`.
4. Work only on those two files. The deletion of your section's template files caused by renaming is permitted. A PR with unrelated changed paths, more than one student directory or both sections is

rejected.

Part 5 - Completing `tokens.md`

1. Read `public.c` from the first character to the last. Ignore whitespace and comments.
2. Record every emitted lexeme in order, with its exact token label from Part 6. Include repeated occurrences. For example, every semicolon gets its own line.
3. Open your `tokens.md`. Replace `STUDENT_ID` and `SECTION` with your real ID and `sec_59_b_and_g`.
4. Replace the placeholder inside the single fenced `text` block with the complete sequence for `public.c`. Keep the fences. Do not add a header row inside the block. This miniature example shows the file format; it is not your complete answer:

```
# Lab 02 token inventory
Student ID: 232-115-001
Section: sec_59_b_and_g

## Token sequence for public.c
```text
int,KEYWORD
total,IDENTIFIER
=,ASSIGNMENT_OPERATOR
25,INTEGER
;,DELIMITER
```
```

5. Keep exactly one fenced block tagged `text`, with three backticks on its opening and closing fence. You may put headings or explanatory notes outside that block.
6. Check your inventory manually before using your program's output to compare it. The full sequence earns 4 marks if it matches exactly; an incorrect or incomplete sequence earns 0. Additional explanations may go outside the block but are not automatically scored.

Part 6 - Completing `lexical_analyzer.c`

1. Open the starter. Keep its argument check and file-open handling, or replace them with equivalent behavior.
2. Use an `int` variable for each value returned by `fgetc`, so you can distinguish a character from `EOF`.
3. Repeatedly read a character. Skip whitespace. If it starts an identifier, gather the entire identifier and then check the keyword list. If it is a digit, gather the whole integer.
4. If you see `/`, look ahead to distinguish line comments, block comments and division. For other operators, check the listed two-character combinations first. Save or put back a look-ahead character that belongs to the next token.
5. Classify delimiters and unsupported single characters. Print the exact token line. Continue until `EOF`, then close the file. Organize repeated work into helper functions if useful.
6. Follow every rule in the contract below. Your scanner does not need to parse expressions, check variable declarations, execute the program or build a symbol table.

The language subset for this lab

Build a lexer for this small, precisely defined C subset. You are not building a full C compiler. Both sections use the same token categories and scanner rules; the source programs differ.

| Token label | Recognize |
|---------------------|---|
| KEYWORD | Only <code>int</code> , <code>if</code> , <code>else</code> , <code>while</code> , <code>return</code> |
| IDENTIFIER | <code>[A-Za-z_][A-Za-z0-9_]*</code> ; a keyword has priority |
| INTEGER | One or more decimal digits: <code>[0-9]+</code> |
| ASSIGNMENT_OPERATOR | <code>=</code> |
| ARITHMETIC_OPERATOR | <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> |
| RELATIONAL_OPERATOR | <code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code> |
| LOGICAL_OPERATOR | <code>&&</code> , <code> </code> , <code>!</code> |
| DELIMITER | <code>(</code> , <code>)</code> , <code>{</code> , <code>}</code> , <code>[</code> , <code>]</code> , <code>;</code> , <code>,</code> |
| UNKNOWN | Any other single non-whitespace ASCII character |

Use case-sensitive matching. For example, `int` is a keyword, but `Int` and `int2` are identifiers. Preserve the original spelling of every lexeme, including leading zeros in integers. A minus sign is a separate operator: `-25` becomes `-`, `ARITHMETIC_OPERATOR` followed by `25`, `INTEGER`.

Skip spaces, tabs, carriage returns and newlines. Skip a `//` comment through newline or end of file. Skip a `/* ... */` comment, including its markers. Block comments do not nest. A slash outside a comment is an arithmetic operator. For an unclosed block comment, print a message to standard error and return a nonzero exit status. Successfully scanned input returns zero.

Choose the longest listed operator before considering one character. For example, `>=` is one relational operator, while `= =` is two assignment operators. The operators `++`, `--`, `+=`, `&` and `|` are not combined tokens in this subset. Scan `++` as two `+` arithmetic operators; scan a lone `&` or `|` as `UNKNOWN`.

Test inputs use ASCII, well-formed identifiers and integers, lexemes of at most 128 characters, and files of at most 64 KiB. Floating-point numbers, exponent notation, hexadecimal numbers, character/string literals, preprocessor directives and C's other keywords are outside the lab subset. These features are not required. When an unsupported character occurs, emit it as `UNKNOWN` and continue. Hidden tests follow this published contract.

Exact output contract

Read the filename from the one command-line argument. Open that file and consume its characters, preferably with `fgetc`. Do not embed a fixed token list and do not read only a line entered at the keyboard.

```
./lexical_analyzer input.c
```

Write exactly one line for each emitted token, in source order:

```
lexeme, TOKEN
```

Do not print a heading, prompt, blank line, debug text, token count, table border or space around the separating comma. End each line with a newline. Diagnostic messages belong on standard error. The grader accepts LF or CRLF line endings and ignores final trailing newlines; other output must match exactly.

For `int total=25;; print:`

```
int, KEYWORD
total, IDENTIFIER
=, ASSIGNMENT_OPERATOR
25, INTEGER
;; DELIMITER
```

The delimiter comma is a literal lexeme. Its exact output has two consecutive commas:

```
,, DELIMITER
```

If the argument is missing, too many arguments are supplied, or the file cannot be opened, write a useful diagnostic to standard error and return a nonzero status. Do not print token output to standard output for those failures.

Part 7 - Testing Locally

1. Make sure GCC is installed, using the Lab 01 setup for your operating system. Run `gcc --version`. On a Mac, use your installed GNU GCC executable, such as `gcc-14`, if `gcc` reports Apple clang. Substitute that GNU GCC command in the commands below.
2. From your section's `lab_02` directory, compile your file. Replace the example ID with your real ID:

```
gcc -std=c11 -Wall -Wextra \
  submissions/232-115-001/lexical_analyzer.c \
  -o lexical_analyzer
./lexical_analyzer public.c
./lexical_analyzer public.c > actual.txt
```

3. On Windows PowerShell, use the single-line compilation and executable command below. Replace `gcc` with your configured GNU GCC executable if necessary:

```
gcc -std=c11 -Wall -Wextra submissions/232-115-001/lexical_analyzer.c -o lexical_analyzer.exe
.\lexical_analyzer.exe public.c
.\lexical_analyzer.exe public.c > actual.txt
```

4. Compare `actual.txt` with the block in your `tokens.md`. Correct errors in either file. Use your reasoning to decide which one is wrong.

5. Create a temporary `input.c` containing the following, and run the analyzer on it. Do not commit this temporary file:

```
int a=25; // ignore this comment
if(a>=2&&a!=0){a=a/5;}
```

6. Check that the comment produces no tokens, `25` stays one integer, `>=`, `&&` and `!=` stay whole operators, and `/` is division. Also try `_item2`, `int2`, a block comment, blank lines, a comma, and a final token without a newline.
7. Run the program with no argument and with a missing file. Verify a nonzero exit and a useful error message, with no decorative standard output. Remove `input.c` and `actual.txt` after testing, or keep them untracked. Binaries are ignored; only your two submission files belong in the PR.

Part 8 - Working Privately Before Submission

1. You may edit and commit locally at any time. From the clone root, stage only your own submission directory and, if renamed, your section's removed `your-id` template files:

```
git add -A sec_59_b_and_g/labs/lab_02/submissions
git diff --cached --name-status
git commit -m "Complete Lab 02 locally"
```

2. Inspect the staged paths before committing. Undo staging for any unrelated path with `git restore --staged PATH`; keep the local file if needed.
3. Do not run `git push` with your solution before the instructor opens submission. Disable or avoid editor actions that automatically publish or sync commits.
4. Keep local work on your own machine until submission time. Your GitHub fork is public, so a push makes your solution visible. The repository cannot technically prevent an early push; this is a course rule.

Part 9 - Submitting the Task

1. Wait for the instructed submission time. Finish your local tests and save both files.
2. Return to the clone root. Run `git status`. If your final edits are uncommitted, stage and commit them as in Part 8. A clean status after an earlier local commit is fine; do not create an unnecessary empty commit.
3. Push your local `main` branch to your student fork:

```
git push origin main
```

4. Open your fork on GitHub. Click **Contribute** then **Open pull request**, or use the upstream repository's **Pull requests > New pull request > compare across forks**.
5. Set the base repository to the instructor's `compiler-demo` and the base branch to `main`. Set the head repository to your student fork and the compare branch to `main`.

6. Examine **Files changed**. The PR must show only your section's two submission files and optional removal of its template. If it contains unrelated files, fix the local changes before submitting.
7. Use this exact title with your real student ID:

```
Lab 02 - sec_59_b_and_g - 232-115-001
```

8. In the description, write your full name, real ID and section. Confirm that you understand the code and disclose AI assistance in one sentence if used. Click **Create pull request**. Keep it open and ready for review, not draft.
9. Send the PR URL through the instructor's announced submission channel. Check the head commit in GitHub and record its full SHA if the instructor asks.
10. Stop pushing when the instructor starts freezing. Do not merge or close the PR yourself. The instructor grades the saved snapshot; later pushes cannot replace it. A PR title/path mismatch or an invalid changed-file set can cause rejection, so fix it before the freeze.

Part 10 - Assignment Submission if the Task Deadline Is Missed

1. Ask the instructor whether you are eligible for the announced assignment extension. Use the same question, same two files, same fork and `main` branch.
2. If you had no task PR, finish, test, commit, push and open the PR using Part 9. If you already had an unfrozen late PR, update that PR instead of opening another PR with the same student ID.
3. Keep the same exact Lab 02 title. Tell the instructor this is an assignment submission. Keep the PR open and stop pushing at the assignment freeze.
4. A student already in the task ledger is skipped completely at assignment freeze, even if they improve their code. Their task grade comes from the task snapshot.
5. The automatic raw mark is out of 20: token inventory 4, successful GCC compilation 2, and seven private executable tests at 2 each. Compilation failure gives 0 for all executable tests, but the inventory can still earn its 4 marks.
6. For a task, the final automatic mark equals the raw mark. For an assignment, the final automatic mark is exactly half the raw mark. For example, $16/20$ becomes $8/20$. Viva notes are recorded separately and do not automatically change this formula.

Part 11 - Viva

1. Bring your own submitted code, token inventory and PR link to the viva.
2. Be ready to trace a new input character by character, explain `EOF` and look-ahead, and show why `int2` is an identifier.
3. Explain how you skip each comment type, how `/` remains division, and how you recognize the longest listed operators.
4. Explain how you used AI, if applicable. You may be asked to modify or debug a small part without copying a new solution. The instructor records explanation notes in the Excel workbook.